

BINGO DARK FANTASY 3.0

Guía de aplicación de POO y SOLID en el código

Proyecto: bingo-dark-fantasy-vue-2.0 (Vue 3 + TypeScript)
Carpeta del código: src/domain, src/application, src/infrastructure
Para qué sirve este documento: explicar al instructor —y recordarme a mí— en qué archivo se aplica cada norma, con ejemplos reales del juego.

1. Idea central (frase para la sustentación)

Dark Fantasy 3.0 se reestructuró en capas. El dominio del bingo (cartón, oráculo, bots, campaña) está modelado con clases, herencia e interfaces (POO). Cada clase tiene una sola responsabilidad; los patrones de victoria y los bots se extienden sin modificar el motor (SOLID). Vue 3 solo presenta ese dominio: no contiene las reglas del juego.

Vue no se convirtió a class Component (eso ya no se usa). Las clases viven en TypeScript, en src/domain. Eso no contradice POO: es el principio D (el dominio no depende de la interfaz).

2. Cómo está organizado el código

Capa	Carpeta	Qué hace
UI	src/views y src/components	Pinta botones, cartones, tienda y lobby. No decide las reglas.
Aplicación	src/application	Una acción del jugador = una clase (comprar, iniciar sesión).
Dominio (POO)	src/domain	El corazón: Carton, Bot, Partida, Campania, Usuario.
Infraestructura	src/infrastructure	localStorage y sessionStorage detrás de interfaces.

Dependencias: las flechas van hacia adentro. Vue usa el dominio. El dominio no importa Vue ni localStorage.

3. POO — dónde está cada pilar

3.1 Encapsulación

Los datos se esconden y se cambian solo con métodos. El cartón no deja marcar un número que no ha cantado el oráculo.

Clase / archivo	Qué encapsula	Métodos que lo protegen
Celda — src/domain/bingo/Celda.ts	número, si es free, si está marcada	marcar(), desmarcar(), toggle()
Carton — src/domain/bingo/Carton.ts	la lista de celdas	marcarNumero(), toggleCelda(), evaluarVictoria()
Bolsa — src/domain/bingo/Bolsa.ts	las bolas restantes	sacar(), quedan()

Clase / archivo	Qué encapsula	Métodos que lo protegen
Oraculo — src/domain/bingo/Oraculo.ts	bolas cantadas y la bolsa	cantar(), yaCanto()
Jugador — src/domain/campania/Jugador.ts	monedas, diamantes, boosters	gastarMonedas(), ganarRecompensa()
Partida — src/domain/partida/Partida.ts	estado idle/playing/won/lost	iniciar(), pausar(), reclamarBingo()

Ejemplo (Celda): marcada es privada. Nadie hace celda.marcada = true desde fuera; hay que llamar marcar().

```
private marcada: boolean
marcar(): void { if (!this.libre) this.marcada = true }
```

3.2 Abstracción

Se define un contrato (qué se puede hacer) sin decir cómo. Las clases concretas llenan el cómo.

Contrato	Archivo	Quién lo implementa
IPatronVictoria	src/domain/bingo/patrones/IPatronVictoria.ts	PatronFila, PatronCruz, PatronCorona...
PatronBase (clase abstracta)	src/domain/bingo/patrones/PatronBase.ts	Todas las formas de victoria
Bot (clase abstracta)	src/domain/bots/Bot.ts	ArzobispoPereza, SantoEspada, BrujaEnvidia, Dragon
Booster (clase abstracta)	src/domain/boosters/Booster.ts	Freeze, Sabotage, GoodLuck, Shield
IRepositorioCampania	src/domain/campania/IRepositorioCampania.ts	LocalStorageCampaniaRepo
IRepositorioSesion	src/domain/identidad/IRepositorioSesion.ts	SessionAuthRepo

3.3 Herencia

Una clase hija reutiliza a la madre y solo cambia lo suyo.

Madre	Hijas	Qué heredan / qué cambian
Bot (abstracta)	ArzobispoPereza, SantoEspada, BrujaEnvidia, Dragon — src/domain/bots/rivales.ts	Heredan decidir() y aplicarMarca(). Cambian nombre, velocidad y fallo.
PatronBase	PatronFila, PatronColumna, PatronDiagonal, PatronL, PatronO, PatronCruz, PatronT, PatronFlecha, PatronH, PatronZigzag, PatronCartonCompleto, PatronCorona — patronesConcretos.ts	Heredan cumple/indices ganadores. Cada una define su forma.
Booster	BoosterFreeze, BoosterSabotage, BoosterGoodLuck, BoosterShield	Heredan el contrato. Cada una tiene precio y moneda.
Usuario	Administrador — src/domain/identidad/Usuario.ts	El admin hereda correo/rol y redefine puedeVerPanel().

```
export abstract class Bot { ... decidir() { /* plantilla */ } }
export class Dragon extends Bot {
  readonly nombre = 'Dragon'
  readonly multVelocidad = 0.7
}
```

3.4 Polimorfismo

Donde el código espera a la madre o a la interfaz, entra cualquiera de las hijas y el juego sigue igual.

- Partida guarda IPatronVictoria[]: puede ser fila o corona sin if (mode === 'row').
- Partida guarda Bot[]: Dragon y Pereza se tratan igual al reaccionar a una bola.
- Campania recibe IRepositorioCampania: hoy es localStorage; mañana podría ser una API.
- Sesion recibe IRepositorioSesion: no sabe si es sessionStorage u otra cosa.

Archivo clave del polimorfismo de patrones: src/domain/bingo/patrones/catalogoPatrones.ts (fábrica modo → clase).

Archivo clave de los rivales: src/domain/bots/FabricaBots.ts.

4. SOLID — dónde está cada letra

S — Single Responsibility (una clase, una razón de cambiar)

Si mañana cambia cómo se cantan las bolas, se toca Oraculo, no Carton. Si cambia el precio de un booster, se toca esa clase, no Partida.

Clase	Una sola responsabilidad	Archivo
Celda	Una casilla	src/domain/bingo/Celda.ts
Carton	El cartón y sus marcas	src/domain/bingo/Carton.ts
Bolsa	Barajar y sacar bolas	src/domain/bingo/Bolsa.ts
Oraculo	Cantar y recordar bolas	src/domain/bingo/Oraculo.ts
DificultadNivel	Tiempos y fallos según el nivel	src/domain/bots/DificultadNivel.ts
Partida	Orquestar una ronda	src/domain/partida/Partida.ts
Campania	Progreso, desbloqueo, victorias	src/domain/campania/Campania.ts
Jugador	Nombre y monedero	src/domain/campania/Jugador.ts
Recompensa	Premio de un nivel	src/domain/campania/Recompensa.ts
CatalogoNiveles	Datos de los 10 niveles	src/domain/campania/CatalogoNiveles.ts
ComprarBooster	El caso de uso comprar	src/application/ComprarBooster.ts
IniciarSesion	El caso de uso login	src/application/IniciarSesion.ts

Vue (LevelPlayView.vue) también queda con una responsabilidad: pintar. usePartida.ts solo traduce la clase Partida a refs de Vue.

O — Open/Closed (abierto a extensión, cerrado a modificación)

Para un patrón nuevo no se edita Partida. Se crea una clase nueva y se registra en la fábrica.

Si quiero...	Qué hago	Qué NO toco
Una forma nueva (estrella)	Crear PatronEstrella extends PatronBase y un case en catalogoPatrones.ts	Partida.ts, Carton.ts, las vistas
Un rival nuevo	Crear class BrujoX extends Bot y añadirlo en FabricaBots	La lógica de marcar bolas
El nivel 11	Un dato en CatalogoNiveles (tamaño, patrones, bolsa)	No copiar Level11View.vue
Un booster nuevo	class BoosterX extends Booster en CatalogoBoosters	La tienda solo lista el catálogo

Los 10 niveles ya no son 10 archivos copiados. Level1View...Level10View son envoltorios de una sola LevelPlayView.vue que lee CatalogoNiveles.

L — Liskov Substitution (la hija sustituye a la madre)

Donde el código espera Bot, puedo pasar Dragon o ArzobispoPereza. Donde espera IPatronVictoria, PatronFila o PatronCorona. Ninguna hija rompe el contrato: todas responden a los mismos métodos.

Se espera	Puede entrar	Por qué no se rompe
Bot	Cualquiera de los 4 rivales	Todos implementan decidir() y toRuntime()
IPatronVictoria	Cualquier patrón concreto	Todos tienen indices() e indicesGanadores()
Usuario	Administrador	Sigue teniendo correo, rol y puedeJugar() / puedeVerPanel()
Booster	Freeze, Shield, etc.	Todos tienen id, nombre, precio, moneda

Ojo de sustentación: Administrador redefine puedeJugar() a false. Sigue el contrato (devuelve boolean); cambia la política. Eso es válido si el contrato dice “indica si puede jugar”, no “siempre puede jugar”.

I — Interface Segregation (interfaces chicas)

Nadie está obligado a implementar métodos que no usa. No hay un “dios” de 20 métodos.

Interfaz	Solo pide	Quién la usa
IPatronVictoria	nombre, índices, ¿ganó?	Carton.evaluarVictoria y Partida
IRepositorioCampania	cargar() y guardar()	Campania — no sabe de login
IRepositorioSesion	guardar, cargar, borrar usuario	Sesion — no sabe de monedas

D — Dependency Inversion (depender de abstracciones)

Las reglas del juego no importan localStorage ni Vue. Reciben una interfaz. Quien crea el objeto concreto es la capa de afuera (store, login, composible).

Quién depende	De qué abstracción	Implementación concreta
Campania	IRepositorioCampania	LocalStorageCampaniaRepo.ts
Sesion / IniciarSesion	IRepositorioSesion	SessionAuthRepo.ts
Partida	IPatronVictoria[] y Bot[]	clases de patrones y rivales
useProgressStore (Pinia)	clase Campania	el store solo sincroniza refs
LevelPlayView.vue	usePartida → Partida	la vista no calcula el bingo

Por eso Vue no “sobra”: es la interfaz. Java sería otro cliente posible sobre el mismo diseño. El dominio ya está separado.

5. Recorrido rápido por archivos (para abrirlos en clase)

Tema	Abre este archivo
Celda y cartón	src/domain/bingo/Celda.ts y Carton.ts
Patrones (O y L)	src/domain/bingo/patrones/IPatronVictoria.ts, PatronBase.ts, patronesConcretos.ts
Fábrica de patrones	src/domain/bingo/patrones/catalogoPatrones.ts

Tema	Abre este archivo
Oráculo	src/domain/bingo/Oraculo.ts y Bolsa.ts
Bots y herencia	src/domain/bots/Bot.ts y rivales.ts
Motor de la ronda	src/domain/partida/Partida.ts
Campaña y D	src/domain/campania/Campania.ts e IRepositorioCampania.ts
Los 10 niveles en datos	src/domain/campania/CatalogoNiveles.ts
Tienda SOLID	src/domain/boosters/Booster.ts y src/application/ComprarBooster.ts
Login POO	src/domain/identidad/Usuario.ts y Sesion.ts
Vue solo pinta	src/views/LevelPlayView.vue y src/composables/usePartida.ts
Persistencia	src/infrastructure/LocalStorageCampaniaRepo.ts

6. Mini mapa de clases (para dibujar en el tablero)

Bingo: Celda → Carton → Bolsa → Oraculo. IPatronVictoria ← PatronFila, PatronCruz, PatronCorona...

Rivales: Bot ← ArzobispoPereza, SantoEspada, BrujaEnvidia, Dragon.

Ronda: Partida usa Carton + Oraculo + Bot[] + IPatronVictoria[].

Campaña: Campania usa Jugador + Nivel (catálogo) + IRepositorioCampania.

Identidad: Usuario ← Administrador. Sesion usa IRepositorioSesion.

Tienda: Booster ← Freeze, Sabotage, GoodLuck, Shield. ComprarBooster usa Campania.

7. Qué decir si preguntan “¿y Vue?”

- Vue 3 usa Composition API, no class components (están deprecados).
- POO está en el dominio TypeScript, no en el .vue.
- Eso es correcto según SOLID D: la UI depende del dominio, no al revés.
- Si el SENA pidiera Java, las mismas clases se traducirían (class Carton, class Dragon extends Bot). El pensamiento ya está.
- No reescribimos el juego a Java porque el producto es una web y ya funciona.

8. Tabla única de normas (resumen para imprimir)

Norma	Dónde se ve en Dark Fantasy
Encapsulación	Celda.marcada privada; Carton.toggleCelda; Jugador.gastarMonedas
Abstracción	IPatronVictoria, Bot abstracta, IRepositorioCampania
Herencia	Dragon extends Bot; Administrador extends Usuario; PatronCruz extends PatronBase
Polimorfismo	Partida trata igual a todos los Bot y a todos los IPatronVictoria
S	Oraculo no guarda monedas; Campania no pinta HTML; ComprarBooster solo compra
O	Patrón o nivel nuevo = clase o dato nuevo, sin editar Partida
L	Cualquier Bot sustituye a Bot; cualquier patrón sustituye a IPatronVictoria
I	Interfaces de 2–4 métodos, no un servicio único gigante

Norma	Dónde se ve en Dark Fantasy
D	Campania no importa localStorage; Vue no calcula el bingo

Documento generado para la sustentación del proyecto Bingo Dark Fantasy 3.0.

Código en TypeScript. Interfaz en Vue 3. Dominio en clases POO + SOLID.